

FerrumMD: Molecular Dynamics in Rust

Main components for Rust developers new to MD

FerrumMD project overview

May 26, 2026

Why this project matters

- **Goal:** build an MD engine with Rust's safety + performance model.
- **Audience bridge:** map familiar systems ideas (data layout, kernels, parallelism) to simulation workflows.
- **Scope today:** short-range interactions, molecular topologies, integrators, thermodynamic controls, and I/O.

MD in one minute

- MD evolves particle positions/velocities by integrating Newton's equations.
- Core loop per step:
 - ① compute forces from current geometry,
 - ② update positions/velocities,
 - ③ apply boundary conditions and controls (thermostat/barostat),
 - ④ report observables and save trajectory.
- Think of it as a repeated `state -> forces -> state'` transform with strict numerical constraints.

High-level FerrumMD architecture

Data and state

`state`, `molecule`, `parameters`, `cell`, `cell_subdivision`

Physics kernels

`lennard_jones`, bonded interactions, electrostatics roadmap, free-energy and quantum modules

Control and workflows

`ensembles` (NVE/NVT/NPT), `thermostat_barostat`, minimization, replica exchange

Interfaces

CLI binaries, GRO/XTC trajectory output, optional viewer feature, optional Python + MPI features

Core simulation state (Rust view)

- Struct-based system representation for particles, molecules, box, and parameters.
- Explicit unit convention (nm, ps, amu, kJ/mol) avoids hidden conversion bugs.
- Rust typing helps separate concerns:
 - immutable parameter sets,
 - mutable evolving state,
 - module boundaries for force kernels and integration.
- Result: easier reasoning about correctness and reproducibility.

Force models implemented

Nonbonded

- Lennard–Jones pair interaction
- Minimum-image convention with periodic boundary conditions
- Cell subdivision module for scaling pair computations

Bonded

- Harmonic bond forces
- Support for small molecular systems (e.g., H_2)
- Topology readers for Martini/CHARMM-style inputs

Time integration and thermodynamic control

- Velocity-Verlet integrator as the default engine.
- Ensemble support:
 - NVE (energy-conserving baseline),
 - pseudo-NVT/NVT with thermostats,
 - NPT path with isotropic barostat support.
- Thermostats implemented: Berendsen and Nose–Hoover.
- Initialization: Maxwell–Boltzmann velocity sampling.

I/O and ecosystem interoperability

- Reads molecular inputs: PDB, GRO, LAMMPS data/dump.
- Force-field conversion helpers: Martini and CHARMM parsers.
- Writes GRO/XTC outputs for external visualization (e.g., VMD).
- Includes a Rust-native viewer scaffold (feature-gated) for extension in pure Rust.
- Python bindings scaffold and MPI-enabled path for integration and parallel demos.

Why Rust is interesting here

- **Safety:** ownership/borrowing reduces hidden aliasing in mutable simulation state.
- **Performance:** predictable data-oriented code, low abstraction penalty.
- **Composability:** feature flags (MPI/viewer/python) keep core minimal.
- **Reliability culture:** strong testing and explicit errors match scientific reproducibility goals.

Roadmap and open opportunities

- Rigid-water constraints (SETTLE/SHAKE-RATTLE path).
- Topology-correct nonbonded exclusions/pairs.
- Electrostatics and units audit for production TIP3P fidelity.
- Possible Rust-focused contributions:
 - SIMD kernel specialization,
 - domain decomposition beyond demo MPI,
 - property-based tests for invariants,
 - GPU offload experimentation.

Takeaways for Rust developers

- ① FerrumMD already demonstrates an end-to-end MD stack in Rust.
- ② The project is a strong playground for systems + scientific computing ideas.
- ③ You can contribute without deep prior MD experience by starting from architecture and tooling modules.

Questions & discussion